

FedSTDN: A Federated Learning-Enabled Spatial-Temporal Prediction Model for Wireless Traffic Prediction

Zhenyu Li, *Graduate Student Member, IEEE*, Yuchuan Fu[✉], *Member, IEEE*, Mengqiu Tian[✉], *Student Member, IEEE*, Changle Li[✉], *Senior Member, IEEE*, F. Richard Yu[✉], *Fellow, IEEE*, and Nan Cheng[✉], *Senior Member, IEEE*

Abstract—Wireless Traffic Prediction (WTP) plays a significant role in achieving intelligent resource management for communication systems. However, WTP still faces challenges such as inaccurate prediction resulting from the complex spatial-temporal characteristics due to user mobility, high communication overhead caused by the complexity of the prediction model, and user privacy issues stemming from Centralized Learning (CL). To address the aforementioned issues, this paper proposes a WTP framework under the Federated Learning (FL) strategy called Federated Spatial-Temporal Dual-attention based Network (FedSTDN). Aiming at improving communication efficiency and simultaneously representing various wireless traffic patterns, a data augmentation-based clustering algorithm is adopted, which groups cells into different regions using a small augmented dataset, facilitating subsequent processing. To improve prediction performance, a local prediction model based on Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) is proposed to capture the short- and long-term dependencies of traffic. Additionally, a novel Kolmogorov-Arnold Network (KAN) layer is introduced to replace the traditional Multi-Layer Perceptron (MLP) layer, further enhancing prediction performance. Simulations on two different real-world datasets verify the effectiveness and efficiency of FedSTDN. Compared to the well-performing baseline, the proposed FedSTDN achieves up to 32.83% and 24.30% improvements in Mean Square Error (MSE) and Mean Absolute Error (MAE) on the Milan dataset, respectively. For the Trentino dataset, FedSTDN achieves up to 17.25% and 5.86% improvements in MSE and MAE, respectively.

Index Terms—Deep learning, federated learning, Kolmogorov-Arnold network, wireless traffic prediction.

Received 10 August 2024; revised 18 February 2025; accepted 4 April 2025. Date of publication 9 April 2025; date of current version 6 August 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62231020, in part by the Innovation Capability Support Program of Shaanxi under Grant 2024RS-CXTD-01, and in part by the Technology Innovation Leading Program of Shaanxi under Grant 2023KXJ-116. Recommended for acceptance by L. Kong. (*Corresponding author: Yuchuan Fu.*)

Zhenyu Li, Yuchuan Fu, Mengqiu Tian, Changle Li, and Nan Cheng are with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, Shaanxi 710071, China, and also with the Research Institute of Smart Transportation, Xidian University, Xi'an, Shaanxi 710071, China (e-mail: ycfu@xidian.edu.cn).

F. Richard Yu is with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada.

Digital Object Identifier 10.1109/TMC.2025.3559066

I. INTRODUCTION

WIRELESS Traffic Prediction (WTP) is an essential component for network operators to perform efficient network management [1]. By adopting appropriate evaluation methods, WTP can provide the future traffic, thereby aiding operators in decision-making [2]. In recent years, thanks to highly reliable real-time communication capabilities and advanced Artificial Intelligence (AI) technologies, WTP has been significantly enhanced in both prediction performance and convergence [3]. AI technology driven by highly reliable transmission and Big Data is expected to become a powerful tool for WTP in Next Generation Network (NGN).

Currently, existing works of WTP employ a Centralized Learning (CL) strategy, which involves uploading massive raw data to data center to train a generalized prediction model [4]. However, frequent transmission of raw data leads to significant communication overhead and negatively impacts communication efficiency. Moreover, uploading raw data also raises concerns about user privacy. Therefore, WTP urgently needs to adopt a new paradigm to address these challenges.

The emergence of Federated Learning (FL) can effectively cope with the aforementioned issues [5], [6]. Under the orchestration of the cloud server, multiple clients (e.g., smartphones, Base Stations (BSs), and Connected Autonomous Vehicles (CAVs) [7]) utilize their local raw data to collaboratively train the global prediction model. After training, only local model parameters or gradient updates are uploaded, which significantly reduces communication overhead and preserves user privacy [8], [9]. Consequently, FL-based WTP has attracted increasing attention from academia and industry. Researchers focus on two main aspects: the prediction performance [10] and the communication overhead of the FL [11], [12]. By capturing the spatial-temporal dependencies and semantic correlations of wireless traffic, the prediction performance can be improved [13]. Through appropriate data processing and aggregation methods, the communication efficiency of FL can be enhanced [14].

Despite considering both aspects of WTP under the FL strategy, it still faces two issues. On the one hand, user mobility makes it difficult to capture and model the spatial-temporal dependencies of wireless traffic across different regions. On the

other hand, designing complex local prediction model may negatively impact the communication efficiency of FL, consuming a large amount of communication resources.

Therefore, to cope with these two issues, this paper proposes a WTP framework called the Federated Spatial-Temporal Dual-attention Network (FedSTDN), through which multiple base stations (BSs) collaboratively train a high-quality prediction model. Compared to other methods, we propose a local model based on Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM), which divides temporal dependencies into short-term and long-term dependencies. A 1D CNN is employed to capture short-term dependencies, while LSTM is used to represent long-term dependencies. Additionally, to further improve model performance, we adopt a Kolmogorov-Arnold Network (KAN) layer. The main contributions of our work are summarized as follows:

- We adopt a data augmentation-based clustering algorithm to group cells into different regions. Specifically, we first apply a data augmentation algorithm to the original traffic data for augmented traffic data. Secondly, we group the cells into several regions based on the augmented traffic patterns and geographic locations using an iterative clustering algorithm. Then, the cloud server uses the shared augmented traffic from each cell to train a quasi-global model and deploys it to the BSs.
- We propose a local prediction model based on CNN and LSTM to effectively capture the short- and long-term dependencies of wireless traffic. Besides, a KAN layer is introduced to replace the traditional Multi-Layer Perception (MLP) layer for performance enhancement.
- We employ a dual-attention based model aggregation algorithm to construct the global model. In particular, we first construct the region-level models by measuring the contributions of cell-level models and region-level models. Then, the global model is obtained by evaluating the contributions of region-level models and the quasi-global model.
- We validate the effectiveness and efficiency of the proposed FedSTDN through two real-world datasets. Additionally, the impact of several key parameters on the prediction performance is analyzed. Compared to the well-performing baseline method, the proposed FedSTDN achieves up to 28.92% and 31.43% improvements in Mean Square Error (MSE), and up to 22.35% and 15.91% improvements in Mean Absolute Error (MAE).

The rest of the paper is as follows. The related works on traffic prediction are presented in Section II. Section III gives the system model and problem formulation. In Section IV, we demonstrate the proposed FedSTDN in detail, including the wireless traffic data augmentation mechanism, the local model and the attention-based aggregation scheme. Simulation and discussion are presented in Section V. Finally, Section VI concludes the paper.

II. RELATED WORK

In this section, we discuss the existing works on traffic prediction. We categorize the existing works into CL strategy and FL strategy.

A. CL Strategy

In recent years, Deep Learning (DL) has strongly supported the CL strategy for WTP, which can well capture nonlinear spatial-temporal dependencies. To capture the entire spatial-temporal dependencies, Qiu et al. [15] employed RNN to capture spatial-temporal correlation between neighboring BSs, and adopted a multi-task learning approach to improve the performance. Besides, Zhang et al. [16] adopted a densely connected CNN to capture the spatial-temporal dependencies of cell traffic by treating traffic data as frame-by-frame images, and proposed a parameter matrix-based fusion scheme to capture spatial-temporal dependencies. In order to improve generalization ability, Zhu et al. [17] proposed an adaptive ensemble learning paradigm that can benefit from centralizing individual forecast base models. Furthermore, an improved CNN-based representation learning method is designed to extract high-order spatial-temporal features. Subsequently, in order to further enhance the prediction performance, the researchers design various network architectures to represent temporal and spatial dependencies separately, and use effective tools such as attention mechanisms to improve accuracy. Wang et al. [18] proposed a Dual-Attention embedded ConvLSTM (DAConvLSTM) network. Shen et al. [19] proposed Time-Wise Attention aided CNN (TWACNet) for WTP. Zheng et al. [20] developed an attention-based ConvLSTM module to extract the spatial and short-term temporal features, where the attention mechanism is properly designed to adaptively allocate different levels of attention to a traffic flow sequence at different times. Mehrabian et al. [21] proposed a dynamic Bernstein graph recurrent network, where a Spectral Dynamic Graph Construction (SDGC) method is designed to model the spatial dependencies while Gated Recurrent Unit (GRU) is used for temporal dependencies. Considering the dynamic spatial dependency, Li et al. [22] proposed a Spatial-Temporal Dynamic Relative-flow Network (STDNRN), where the Relative-flow Gating Mechanism (RGM) combined with CNN is designed for the dynamic spatial dependencies, and the LSTM is used to represent the temporal dependencies. Additionally, Yu et al. [23] and Sun et al. [24] proposed a fine-grained neural network and a Graph-based Temporal Convolutional Network (GTCN) to predict the future traffic of each network unit in a wireless network.

Under the CL strategy, due to the centralized data, WTP can leverage rich raw data for training, achieving higher model accuracy and generalization capability. However, uploading users' raw data involves privacy concerns, and a large amount of data transmission to the cloud server incurs significant communication overhead. Therefore, it is necessary to adopt the FL strategy to further reduce communication overhead while protecting user privacy.

B. FL Strategy

In this paper, the existing works on traffic prediction under the FL strategy can be divided into two aspects: prediction performance and communication overhead.

In order to improve prediction performance, Zhang et al. [10] trained a sensitive initial model that can quickly adapt to

heterogeneous cells. A distance-based weighted model aggregation scheme was proposed to further characterize the spatial-temporal dependencies. Additionally, Li et al. [25] proposed a split learning-based FL architecture for network traffic prediction. Each client trains a dimension-specific prediction model using local dataset, and the prediction models are aggregated through a collaborative interaction process. Shen et al. [26] proposed a Decentralized FL Spatial-Temporal Transformer Network (DeFedSTTN). In DeFedSTTN, a spatial-temporal transformer network was introduced in the local model to capture the dynamic spatial-temporal dependencies among urban regions. Besides, Qi et al. [27] proposed a DL framework based on FL and Asynchronous GCN (FedAGCN) to predict traffic flow, which applies asynchronous spatial-temporal graph convolution to capture the spatial-temporal dependencies. Having considered the semantic correlation, Liu et al. [28] proposed a Multi-level FL framework (MFL), which reconstructs the adjacency matrix dynamically.

To reduce communication overhead, Mobile Edge Computing (MEC) significantly enhances the efficiency and application scope of FL by providing a distributed, low-latency, and privacy-preserving computing environment [29], [30]. Solat et al. [31] proposed a FL scheme for mobile traffic prediction. The convergence rate is further improved by optimizing the creation of groups and their association with MEC servers. In addition, Kim et al. [32] proposed a novel joint MEC server selection and dataset management mechanism for FL-based mobile traffic prediction over MEC servers, including an optimization problem for balancing the accuracy-cost tradeoff. Phyu et al. [33] proposed the Federated Proximal LSTM (FPLSTM) framework, the performance of which is as accurate as the CL strategy, while ensuring computation and communication efficiency.

The existing works have made great efforts in prediction performance and communication overhead of traffic prediction under FL strategy. However, due to complex spatial-temporal dependencies caused by user mobility and data heterogeneity from different BSs, there is still room for improvement in prediction performance. Besides, the high complexity of local prediction model may also affect communication efficiency. To address these issues, this paper proposes FedSTDN.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first present the problem formulation for WTP. Additionally, we provide the detailed preliminaries of FL corresponding to the formulated problem.

A. Problem Formulation

In this paper, we consider a city composed of $H \times W$ cells, where H and W represent the height and width of the city, respectively. Additionally, there are $H \times W$ base stations (BSs), with each BS responsible for one cell, and each BS has its own local wireless traffic data, denoted as $\mathbf{d}_i = \{d_i^1, d_i^2, \dots, d_i^t, \dots, d_i^T\} \in \mathbb{R}^{1 \times T}$, where d_i^t represents the traffic in the cell i at time interval t , and T denotes the total number of time intervals. Based on historical traffic data, WTP is a typical spatial-temporal prediction problem. In this paper,

we focus on the one-step prediction problem. Assuming d_i^{t+1} is the target wireless traffic to be predicted, the WTP problem can be formulated as

$$\hat{d}_i^{t+1} = f(d_i^1, d_i^2, \dots, d_i^t | \omega), \quad (1)$$

where $f(\cdot)$ denotes the prediction model and ω the corresponding parameters.

In terms of machine learning (ML) based WTP techniques, it is common to utilize only part of the historical traffic data as input features to reduce the computation complexity. Thus, based upon $\mathbf{d}_i$, we can obtain a series of input-output pairs $\{\mathbf{x}_i^k, y_i^k\}_{k=1}^u$ through the sliding window scheme [34]. $\mathbf{x}_i^k$ denotes the historical traffic data related to y_i^k , and it is set to $\{d_i^t, d_i^{t-1}, \dots, d_i^{t-p}, d_i^{t-\Phi+1}, d_i^{t-\Phi+2}, \dots, d_i^{t-\Phi+q}\}$, where p and q are the sliding window sizes for capturing the closeness dependence and period dependence of wireless traffic data, and Φ denotes the periodicity [16]. Therefore, we establish the problem based on (1) as

$$\hat{d}_i^{t+1} = \hat{y}_i^k = f(\mathbf{x}_i^k; \omega). \quad (2)$$

In this paper, our goal is to minimize the prediction error over all $H \times W$ BSs, thus the parameters ω can be obtained by solving

$$\arg \min_{\omega} \left\{ \frac{1}{(H \times W)u} \sum_{i=1}^{H \times W} \sum_{k=1}^u \text{Loss}(f(\mathbf{x}_i^k; \omega), y_i^k) \right\}, \quad (3)$$

where $\text{Loss}(\cdot)$ is the loss function. u denotes the number of input-output pairs used in the sliding window scheme.

B. Preliminaries of FL

FL is a distributed ML approach, which enables a group of clients to collaboratively train a global model without data exchange [6]. In the context of WTP for a city, the clients refer to cell-level clients within the city area, and the data of each cell is only stored in the local database. The training procedures of FL at time t are outlined as follows:

- *Initialization*: The cloud server send the global model ω_k^t to all BSs.
- *Local training*: The BS treats the global model as prior knowledge and updates its local model based upon ω_k^t with its local dataset. The local model update rule is given as follows

$$\omega_k^{t+1} = \omega_k^t - \eta \nabla_{\omega_k^t} \text{Loss}(f(\mathbf{x}_i^k; \omega_k^t), y_i^k), \quad (4)$$

where η is the learning rate and $\nabla_{\omega_k^t}$ is the gradient of loss function with respect to ω_k^t .

- *Parameter aggregation*: The BS sends ω_k^{t+1} to the cloud server, and then the cloud performs model aggregation (a.k.a. federated optimization) based on the local models uploaded to the cloud.

By running the above steps iteratively until the termination conditions are satisfied, the final global model ω_k can be obtained.

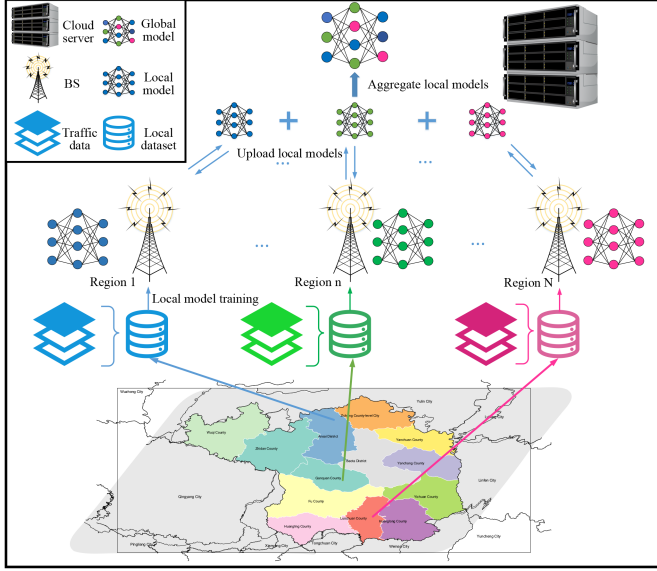


Fig. 1. The system diagram of FedSTDN.

IV. METHODOLOGY

In this section, we provide a detailed introduction of the proposed FedSTDN, and the system diagram of FedSTDN is shown in Fig. 1.

A. Wireless Traffic Data Augmentation Mechanism

Urban areas have different functions to support the daily operation of a city, leading to significant variations in traffic patterns among different BSs [35]. Additionally, users exhibit diverse mobility and communication behaviors, further increasing the diversity of wireless traffic patterns. As a result, the wireless traffic data from different BSs is highly heterogeneous and essentially non-IID. Performing FL on non-IID data is quite challenging due to weight divergence during model aggregation at the cloud side [36]. To address the issue of statistical heterogeneity, we employ a data-sharing strategy by constructing a small augmentation dataset using the original wireless traffic data.

In terms of data augmentation, we provide the visualization of the data augmentation mechanism [14] in Fig. 2. To begin with, we split the original wireless traffic data into weekly slices, each containing 7 days. Next, within each slice, the values corresponding to the same hour index are aggregated across different days by calculating the statistical average at each hour to obtain augmented traffic values. After that, we concatenate the augmented traffic of each slice in week index order and standardize the augmented traffic data to have zero mean and unit variance, resulting in the required augmented traffic dataset. The specific process is illustrated in Algorithm 1.

B. STN

We propose the STN-based personalized learning as the local prediction model, which consists of the CNN and LSTM. With the aid of STN, cells can capture the short-term and long-term

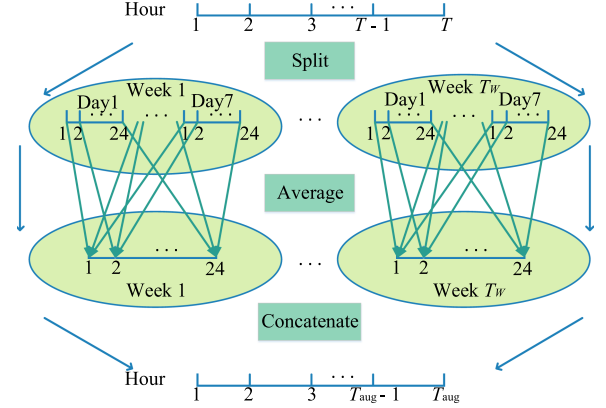


Fig. 2. The visualization of wireless traffic data augmentation mechanism.

Algorithm 1: Wireless Traffic Data Augmentation Mechanism.

Input: The set of wireless traffic data $\left\{\{d_i^t\}_{i=1}^{H \times W}\right\}_{t=1}^T$.

$\tilde{\mathbf{d}} = \{\}$

for $(i = 1; i \leq H \times W; i++)$ **do**

Split $\{d_i^t\}_{t=1}^T$ into weekly slices according to time index as $\{\mathbf{d}_i^w\}_{w=1}^{T_W}$.
 $(T_W$ denotes the number of weekly slices.)

Aggregate $\{\mathbf{d}_i^w\}_{w=1}^{T_W}$ by hour index to obtain the aggregated traffic data $\{d_i^{t'}\}_{t'=1}^{T_{\text{aug}}}$.
 $(T_{\text{aug}}$ denotes the total number of time indexes after aggregation.)

$\tilde{\mathbf{d}}.\text{add}(\{d_i^{t'}\}_{t'=1}^{T_{\text{aug}}})$

end

return $\tilde{\mathbf{d}}$

dependencies from their local wireless traffic data. As illustrated in Fig. 3, STN mainly consists of three blocks: Short-term dependency capture block, Long-term dependency capture block, and Prediction output block. The short-term and long-term dependency capture block are used to capture the underlying local features and periodic information of wireless traffic data. The procedure of STN is summarized in Algorithm 2.

1) *Short-Term Dependency Capture Block:* After the cloud obtains the augmented data, an iterative clustering strategy is first adopted to group cells into regions, which have similar traffic patterns as well as immediate geographical location information. The specific procedure of the iterative clustering strategy is illustrated in Algorithm 3. Aiming at capturing the short-term dependencies of wireless traffic, we utilize 1D CNN to learn the local features.

2) *Long-Term Dependency Capture Block:* Concerning the long-term dependencies, we utilize LSTM [37] to capture the periodic features shown in Fig. 3, expressed as

$$\mathbf{x}^{(l-1)} = \text{Concat}(\mathbf{d}_i^1, \dots, \mathbf{d}_i^T, \dots, \mathbf{d}_N^1, \dots, \mathbf{d}_N^T), \quad (5)$$

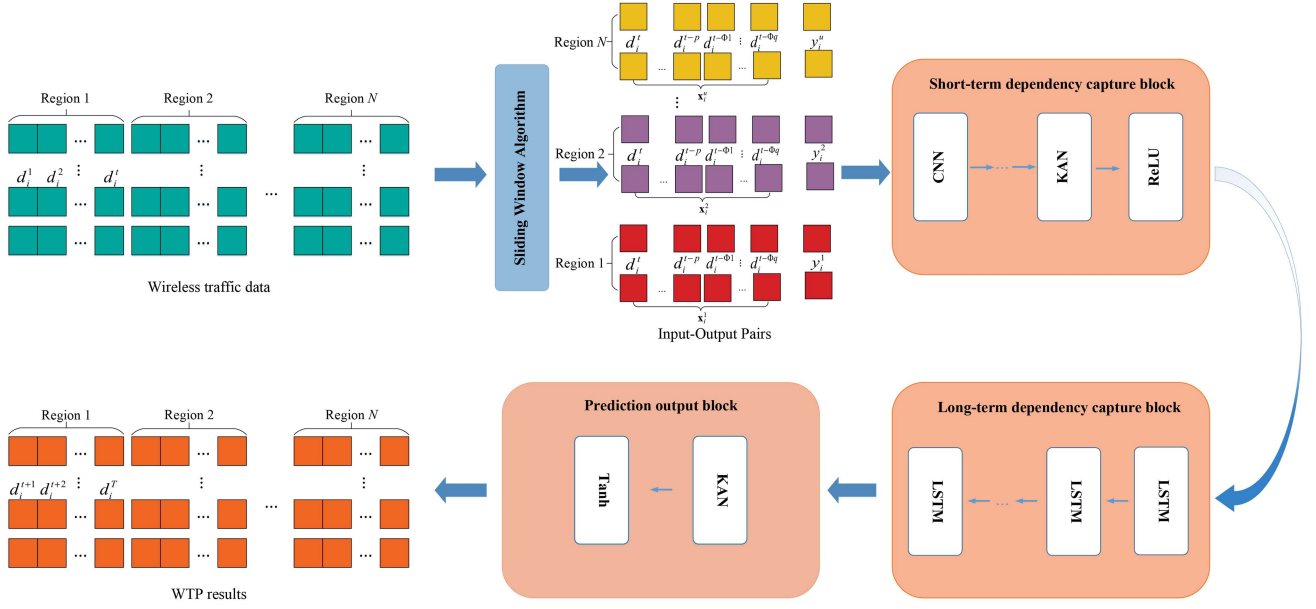


Fig. 3. The architecture of local STN model.

Algorithm 2: The Implementation of STN.

Input: The set of Input-Output pairs $\left\{ \left[(\mathbf{x}_i^k, y_i^k)_{k=1}^u \right]_{i=1}^N \right\}$, the number of the past time intervals N_P , the number of regions N , the number of layers for CNN K , the number of LSTM cells N_L .

```

g = {}, l = 1
for (i = 1; i <= N; i++) do
    for (t = 1; t <= N_P; t++) do
         $d_i^{t,(0)} = d_i^t$ 
        for (c = 1; c <= K; c++) do
             $d_{conv}^{t,(c)} = \text{ReLU}(\text{Conv1d}(d_i^{t,(c-1)}))$ 
             $d_i^{t,(c)} = d_{conv}^{t,(c)}$ 
        end
         $\mathbf{g}_i^{t,flat} = \text{Flatten}(d_i^{t,(K)})$ 
         $\mathbf{g}_i^t = \text{KAN}(\mathbf{g}_i^{t,flat})$ 
        g.add( $\mathbf{g}_i^t$ )
    end
end
 $\mathbf{g}^{(0)} = \mathbf{g}$ 
while (l <=  $N_L$ ) do
     $\mathbf{h}^{(l)} = \text{LSTM}(\mathbf{g}^{(l-1)})$ 
     $\mathbf{g}^{(l)} = \mathbf{h}^{(l)}$ 
    l = l + 1
end
 $\hat{\mathbf{y}} = \text{KAN}(\mathbf{h}^{(N_L)})$ 
return  $\hat{\mathbf{y}}$ 

```

Algorithm 3: Iterative Clustering Strategy.

Input: BS location $\{g_k^t\}_{k=1}^{H \times W}$, augmented traffic data $\tilde{\mathbf{d}}$, the number of regions N , region center $\{v_i^t\}_{i=1}^N$, and iteration threshold J .

Initialize $\{v_i^t\}_{i=1}^N$

```

for (j = 1; j < J; j++) do
    Group  $\tilde{\mathbf{d}}$  into  $N$  regions  $\{l_i^t\}_{i=1}^N$  through K-Means with  $\{v_i^t\}_{i=1}^N$ 
    Group  $\{g_k^t\}_{k=1}^{H \times W}$  into  $N$  regions  $\{l'_i\}_{i=1}^N$  through K-Means
    if  $\{l_i^t\}_{i=1}^N = \{l'_i\}_{i=1}^N$  then
        break
    end
    Update  $\{v_i^t\}_{i=1}^N$  based on  $\{l'_i\}_{i=1}^N$ 
end
return  $N$  regions

```

$$\mathbf{c}^{(l)} = \mathbf{f}^{(l)} \times \mathbf{c}^{(l)} + \mathbf{e}^{(l)} \times g(\mathbf{W}_{cx}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{W}_{hh}^{(l)} \mathbf{h}^{(l)} + \mathbf{W}_{cc}^{(l)} \mathbf{c}^{(l)} + \mathbf{b}_c^{(l)}), \quad (8)$$

$$\mathbf{o}^{(l)} = \text{Sigmoid}(\mathbf{W}_{ox}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{W}_{hh}^{(l)} \mathbf{h}^{(l)} + \mathbf{W}_{oc}^{(l)} \mathbf{c}^{(l)} + \mathbf{b}_o^{(l)}), \quad (9)$$

$$\mathbf{h}^{(l)} = \mathbf{o}^{(l)} \times \mathbf{h}(\mathbf{c}^{(l)}), \quad (10)$$

where $\times$ represents the scalar product of two vectors or matrices, $g(\cdot)$ and $\mathbf{h}(\cdot)$ are extensions of the standard sigmoid function, and the range becomes $[-2, 2]$ and $[-1, 1]$ respectively. $\mathbf{h}^{(l)}$ denotes the output of l -th LSTM cell.

3) *Prediction Output Block*: The prediction output block consists of a KAN layer and a Tanh activation function, designed

$$\mathbf{e}^{(l)} = \text{Sigmoid}(\mathbf{W}_{ex}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{W}_{hh}^{(l)} \mathbf{h}^{(l)} + \mathbf{W}_{ec}^{(l)} \mathbf{c}^{(l)} + \mathbf{b}_e^{(l)}), \quad (6)$$

$$\mathbf{f}^{(l)} = \text{Sigmoid}(\mathbf{W}_{fx}^{(l)} \mathbf{x}^{(l-1)} + \mathbf{W}_{hh}^{(l)} \mathbf{h}^{(l)} + \mathbf{W}_{fc}^{(l)} \mathbf{c}^{(l)} + \mathbf{b}_f^{(l)}), \quad (7)$$

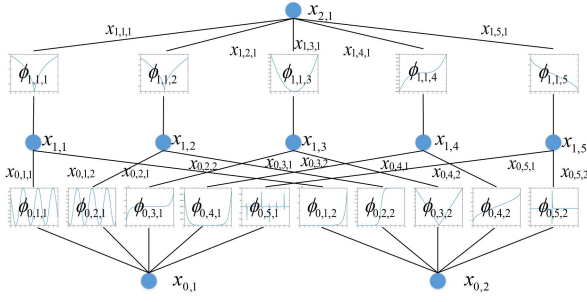


Fig. 4. The structure of three-layer KAN.

to learn the spatial-temporal features obtained from the short-term dependency capture block and the long-term dependency capture block. Based on [38], KAN has no linear weights at all, as each weight parameter is replaced by a univariate function parameterized as a spline. This enables smaller KAN to achieve comparable or even better performance than larger MLP in function fitting tasks. Therefore, we adopt KAN layer to replace traditional MLP layers, resulting in better performance. For the KAN layer, it is based on the Kolmogorov-Arnold representation theorem [39], which can be described as follows:

If f is a multivariate continuous function on a bounded domain, then f can be written as a finite composition of continuous functions of a single variable and the binary operation of addition. More specifically, for a smooth $f : [0, 1]^n \rightarrow \mathbb{R}$,

$$f(\mathbf{x}) = f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (11)$$

where $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$.

Inspired by the aforementioned theorem, researchers are not limited to using (11) with only two-layer nonlinearities and a small number of terms ($2n + 1$) in the hidden layer. Instead, they aim to further extend the network to arbitrary width and depth. Assume we have a supervised learning task composed of input-output pairs $\{x_i, y_i\}$, where we aim to find a function f such that $y_i \approx f(x_i)$ for all data points. (11) indicates that if we find appropriate univariate functions $\phi_{q,p}$ and Φ_q , the task can be accomplished. This inspires us to design a neural network that explicitly parameterizes (11). Since all functions to be learned are univariate functions, we can parameterize each one-dimensional function as a B-spline curve with learnable local B-spline basis function coefficients, as referenced in [38].

In Fig. 4, we present the structure of the KAN layer. As can be seen from the figure, this is a three-layer KAN structure, consisting of an input layer, a hidden layer, and an output layer. The nodes in each layer are connected to the nodes of the next layer via activation functions on the edges. Here, $\phi_{l,q,p}$ represents the activation function from the p -th neuron of the l -th layer to the q -th neuron of the $(l + 1)$ -th layer. In light of this, a KAN layer with n_{in} -dimensional inputs and n_{out} -dimensional outputs can be defined as a matrix of 1D functions

$$\Phi = \{\phi_{q,p}\}, q = 1, 2, \dots, n_{out}, p = 1, 2, \dots, n_{in}, \quad (12)$$

where each activation function $\phi_{q,p}$ is a univariate function that maps a single input value to an output value. These functions are parameterized using splines, which are piecewise polynomial functions. The spline functions are defined over a set of grid points, and their coefficients are learnable parameters.

Given an input vector $\mathbf{x} \in \mathbb{R}^{n_{in}}$, the output of the KAN layer is computed by summing the outputs of the activation functions applied to each input element. Specifically, the activation value of the q -th neuron in the output layer is computed as:

$$x_{out,q} = \sum_{p=1}^{n_{in}} \phi_{q,p}(x_{in,p}), \quad (13)$$

where $x_{in,p}$ is the p -th element of the input vector, and $\phi_{q,p}$ is the activation function associated with the edge from the p -th input neuron to the q -th output neuron. Each activation function $\phi_{q,p}$ is parameterized as a spline. The spline is defined over a series of grid points, and the coefficients of the spline basis functions are learnable parameters. The spline function can be expressed as

$$\phi_{q,p}(x_{in,p}) = \sum_j c_{q,p,j} B_j(x_{in,p}), \quad (14)$$

where $B_j(\cdot)$ denotes the B-spline basis function, and $c_{q,p,j}$ is learnable.

Although (13) looks simple, it is non-trivial to make it well optimizable. To solve this, residual activation functions are employed. Specifically, each KAN layer includes a residual component in each activation function

$$\phi_{q,p}(x_{in,p}) = \omega_b b(x_{in,p}) + \omega_s \sum_j c_{q,p,j} B_j(x_{in,p}), \quad (15)$$

where $b(\cdot)$ is a basis function, such as SiLu function, and ω_b and ω_s are learnable scaling factors. The KAN layer is trained using backpropagation, where the gradients of the loss function with respect to the spline coefficients are computed and used to update the parameters.

Therefore, given the spatial-temporal features of the traffic after being processed by the short- and long-term dependency capture block, the output of the prediction output block is

$$\begin{aligned} \hat{\mathbf{y}} &= \text{Tanh}(\Phi_{out}) \mathbf{h}^{(N_L)} \\ &= \text{Tanh} \left(\sum_{i_{in}=1}^{n_{in}} \phi_{i_{out},i_{in}}(\mathbf{h}^{(N_L)}) \right). \end{aligned} \quad (16)$$

C. Attention-Based Model Aggregation

The last step of FL is the model aggregation at the cloud server, which constructs the final global model from the received local models [40], [41]. In this paper, we adopt an attention-based model aggregation algorithm to obtain the global model. In particular, a dual attention mechanism is introduced into the model aggregation process, assessing the contributions of both local models and a quasi-global model in a layer-wise manner. Fig. 5 provides a visual illustration of the dual-attention based model aggregation method. FedSTDN employs a hierarchical

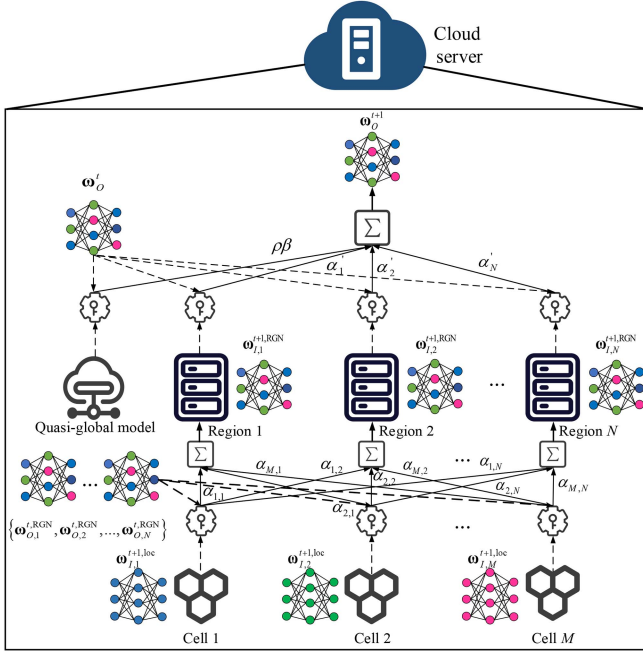


Fig. 5. The model aggregation of FedSTDN.

learning approach with two tiers of model aggregation, which consists of the cell-level and the region-level model aggregation.

The optimization objective is to minimize the sum-weighted distance among different models by using self-adaptive scores as weights. The federated optimization problem is formally defined as

$$\arg \min_{\omega_{O,n}^{t+1,\text{RGN}}} \left\{ \sum_{m=1}^M \frac{1}{2} \alpha_{m,n} \left(\mathcal{L} \left(\omega_{O,n}^{t,\text{RGN}}, \omega_{I,m}^{t+1,\text{loc}} \right) \right)^2 \right\}, \quad (17)$$

$$\arg \min_{\omega_O^{t+1}} \left\{ \sum_{n=1}^N \frac{1}{2} \alpha'_n \left(\mathcal{L} \left(\omega_O^t, \omega_{I,n}^{t+1,\text{RGN}} \right) \right)^2 + \frac{1}{2} \rho \beta \left(\mathcal{L} \left(\omega_O^t, \omega_Q \right) \right)^2 \right\}, \quad (18)$$

where $\omega_{I,m}^{t+1,\text{loc}}$ denotes the m -th local model trained for each cell, and there are M input models in total. $\omega_{O,n}^{t+1,\text{RGN}}$ denotes the output model of each BS, and there are N regions in total. The final output is the global model ω_O^{t+1} . In addition, $\alpha_{m,n}$, α'_n and β denote attention weight vectors representing the contributions of the cell-level layer, region-level layer, and the quasi-global model; ρ is a task-dependent regularization parameter which is set initially. Here we will study how to obtain $\alpha_{m,n}$, α'_n and β .

To determine the weights $\alpha_{m,n}$, α'_n , we employ the attention mechanism and apply it to the parameters of each layer. Let $\omega_{I,m}^{\text{loc}}$ represent the parameter of the first layer for the m -th input model, and $\omega_{O,n}^{\text{RGN}}$ represent the parameter of the first layer for the output model. $\omega_{I,n}^{\text{RGN}}$ and ω_O denote the parameter of the second layer for the n -th input model and the parameter of the second layer for the output model, i.e., global model. For simplicity, we omit the time stamps in those model parameters. The distance between the input and output

model for each layer can then be calculated using the Frobenius norm of their difference, which is expressed as

$$d_{m,n}^{(1)} = \mathcal{L} \left(\omega_{I,m}^{\text{loc}}, \omega_{O,n}^{\text{RGN}} \right) = \left\| \omega_{I,m}^{\text{loc}} - \omega_{O,n}^{\text{RGN}} \right\|_F, \quad (19)$$

$$d_n^{(2)} = \mathcal{L} \left(\omega_{I,n}^{\text{RGN}}, \omega_O \right) = \left\| \omega_{I,n}^{\text{RGN}} - \omega_O \right\|_F. \quad (20)$$

Subsequently, the softmax function is applied to $d_{m,n}^{(1)}$ and $d_n^{(2)}$, which maps the non-normalized distance values to a probability distribution over the input models. In this way, the contributions of these models can be expressed as

$$\alpha_{m,n} = \frac{e^{d_{m,n}^{(1)}}}{\sum_{m=1}^M e^{d_{m,n}^{(1)}}}, \alpha'_n = \frac{e^{d_n^{(2)}}}{\sum_{n=1}^N e^{d_n^{(2)}}}. \quad (21)$$

The weight β can be obtained in the same way. After we get the $\alpha_{m,n}$, α'_n and β , the output model parameters can be updated through the Stochastic Gradient Descent (SGD) algorithm. We calculate the derivation of (17),(18) with respect to $\omega_{O,n}^{t,\text{RGN}}$ and ω_O^t . Then we obtain the corresponding gradient

$$\nabla_n^{(1)} = \sum_{m=1}^M \alpha_{m,n} \left(\omega_{O,n}^{t,\text{RGN}} - \omega_{I,m}^{t+1,\text{loc}} \right), \quad (22)$$

$$\nabla^{(2)} = \sum_{n=1}^N \alpha'_n \left(\omega_O^t - \omega_{I,n}^{t+1,\text{RGN}} \right) + \rho \beta \left(\omega_O^t - \omega_Q \right). \quad (23)$$

With the derived gradient, the output model parameters of each layer can be updated by

$$\omega_{O,n}^{t+1,\text{RGN}} = \omega_{O,n}^{t,\text{RGN}} - \gamma \left(\sum_{m=1}^M \alpha_{m,n} \left(\omega_{O,n}^{t,\text{RGN}} - \omega_{I,m}^{t+1,\text{loc}} \right) \right), \quad (24)$$

$$\omega_O^{t+1} = \omega_O^t - \gamma \left(\sum_{n=1}^N \alpha'_n \left(\omega_O^t - \omega_{I,n}^{t+1,\text{RGN}} \right) + \rho \beta \left(\omega_O^t - \omega_Q \right) \right), \quad (25)$$

where γ denotes the predetermined step size.

V. SIMULATION AND DISCUSSION

In this section, extensive simulations are conducted to validate the effectiveness and efficiency of FedSTDN. Firstly, we provide a brief introduction of the dataset, evaluation metrics, and baselines. Secondly, the experimental settings are given. Thirdly, we analyze the simulation results, including the overall prediction performance of various methods and the effect of key parameters on prediction performance.

A. Dataset and Evaluation Metrics

In this paper, we select the wireless traffic dataset from the city of Milan and the province of Trentino. The dataset is from the ‘‘Big Data Challenge’’ [42] sponsored by Telecom Italia. The area of Milan is divided into a grid of $H \times W$ (100×100) squares and each square is referred to as a cell, which covers an area of 235 m $\times$ 235 m. The area of Trentino is divided into a grid of 6575 cells. Among them, we mainly use seven types of data:

Date, Cell ID, Longitude and Latitude of each cell, the wireless traffic of SMS, Call, and Internet. The traffic is logged every ten minutes over two months, from 11/01/2013 to 01/01/2014, and we fill the missing values by calculating the average of the values before and after the missing data points. For the simulations in the following subsections, the traffic is aggregated into an hour scale and is scaled into [0,1] via Min-Max normalization. Furthermore, we adopt two widely used evaluation metrics in this paper, i.e., MSE and MAE, which are expressed as

$$\text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2, \quad (26)$$

$$\text{MAE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{N} \sum_{n=1}^N |y_n - \hat{y}_n|, \quad (27)$$

where $\mathbf{y} = \{y_1, y_2, \dots, y_N\}$ and $\hat{\mathbf{y}} = \{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N\}$ denote the groundtruth and the prediction of the wireless traffic.

B. Baselines

The proposed FedSTDN is compared with the following baseline methods.

- *STDRN* [22]: The WTP method under CL strategy, which considers both spatial and temporal dependencies of wireless traffic.
- *FedAvg* [5]: The pioneering work in FL first introduced FedAvg, which aggregates the model by averaging the local weights.
- *FedAtt* [43]: This algorithm is similar to FedAvg. However, during model aggregation on the central server, it distinguishes and quantifies the contributions of each client model to the global model.
- *FedDA* [14]: This algorithm further proposed a dual attention scheme by aggregating the intra- and inter-cluster models, instead of simply averaging the weights of local models.
- *FedMLP*: The proposed algorithm without KAN.

C. Parameter Settings and Experiment Analysis

In our experiment, Python 3.9 and Torch 1.12.0 are utilized as the simulation environment. We randomly select 100 cells from each dataset to record three types of wireless traffic. The traffic data from the first seven weeks was used to train the prediction model, while the traffic data from the eighth week was used for testing. We adopt the sliding window method to construct training samples. According to [14], [38] and [44], we summarize the simulation parameters as shown in the Table I. In each communication round, 10% of the cells participate in model training. We consider SGD as the optimizer to update our models.

The experimental results of different prediction methods are shown in Table II. It should be noted that in Table II, FedDA, FedMLP, and our proposed method have three variants based on the number of shared enhanced data samples, namely $\varphi = 1$, $\varphi = 10$, and $\varphi = 100$. In practice, the number of transmitted data samples can be flexibly adjusted according to the network

TABLE I
PARAMETER SETTINGS

Description	Value
The length of closeness dependence p	3
The length of periodicity dependence q	3
The number of hidden neurons for LSTM	64
The kernel size of CNN	3
The number of communication rounds	100
The task-dependent regularizer ρ	[-0.3, 0.3]
The number of regions R	16
The number of local epochs	1
The local batchsize	20
Learning rate	0.01
The number of grid intervals G	5
The order of piecewise polynomial v	3
The grid range	[0.5, 1.2]

conditions. As can be seen from Table II, our proposed method FedSTDN outperforms all baseline methods in all types of wireless traffic and across both datasets, even when only 1% of augmented data is shared. In this and the following experiments, unless otherwise stated, the results reported are those of FedSTDN with $\varphi = 100$. Specifically, for the Milan dataset, FedSTDN achieves up to 61.03%, 38.91%, 38.91%, 50.95%, and 32.83% improvement in MSE, and up to 41.80%, 26.65%, 26.76%, 34.79%, and 24.30% improvement in MAE compared to STDRN, FedAvg, FedAtt, FedDA, and FedMLP, respectively. Similarly, for the Trentino dataset, FedSTDN provides up to 72.99%, 45.84%, 45.10%, 33.18%, and 17.25% improvement in MSE, and up to 29.17%, 18.91%, 18.62%, 13.34%, and 5.86% improvement in MAE, respectively. We also note that the prediction performance of FedDA, FedMLP, and FedSTDN continues to improve with an increase in the amount of shared augmented data. This is because when more data samples are available, the quasi-global model can better capture traffic patterns. The reasons for the superiority of FedSTDN over other baseline methods are as follows:

- 1) Compared to the STDRN method based on the CL strategy, which may be affected by uneven data distribution and computational bottlenecks, the FL strategy effectively leverages distributed data and computing resources across devices, adapting to the characteristics of different devices and regions. As a result, FedSTDN performs better on the two datasets.
- 2) Compared with traditional FedAvg and FedAtt algorithms, FedSTDN introduces a clustering strategy to make the patterns of wireless traffic more specific, the local model can characterize short-term and long-term dependencies, and the dual attention mechanism greatly reduces the heterogeneity of data. Therefore, FedSTDN has a higher generalization ability.
- 3) Compared with the advanced FedDA, the CNN is considered in the local model, which enables FedSTDN to characterize the short-term dependencies of clustered traffic data, thereby improving the prediction performance. In addition, the KAN layer is introduced to replace the fully connected layer, so that the weight activation replaces the

TABLE II
COMPARISON OF MSE AND MAE FOR DIFFERENT METHODS ON TWO DATASETS

Methods	Milan						Trentino					
	MSE			MAE			MSE			MAE		
	Call	SMS	Internet	Call	SMS	Internet	Call	SMS	Internet	Call	SMS	Internet
STDRN	0.1032	0.4199	0.1271	0.2174	0.3791	0.2596	1.1213	3.6411	4.6295	0.4934	0.9278	1.1017
FedAvg	0.0776	0.3744	0.1096	0.1838	0.3386	0.2319	1.6048	2.2287	4.7988	0.5319	0.7416	1.0688
FedAtt	0.0774	0.3667	0.1096	0.1837	0.3375	0.2321	1.5967	2.1558	4.7645	0.5306	0.7444	1.0629
FedDA($\varphi = 1$)	0.0794	0.3798	0.1191	0.1841	0.3537	0.2468	1.4655	2.2940	4.7068	0.5068	0.7452	1.0478
FedDA($\varphi = 10$)	0.0790	0.3748	0.1175	0.1856	0.3503	0.2438	1.4299	2.2364	4.5254	0.5023	0.7351	1.0148
FedDA($\varphi = 100$)	0.0780	0.3381	0.1110	0.1828	0.3468	0.2358	1.3817	2.1737	4.2412	0.4918	0.7216	0.9737
FedMLP($\varphi = 1$)	0.0696	0.3439	0.1071	0.1739	0.3335	0.2307	1.2244	2.5104	3.8752	0.4659	0.7685	0.9490
FedMLP($\varphi = 10$)	0.0694	0.3437	0.1058	0.1720	0.3317	0.2282	1.1923	2.5439	3.8718	0.4622	0.7756	0.9433
FedMLP($\varphi = 100$)	0.0671	0.3291	0.1048	0.1715	0.3308	0.2276	1.1026	2.4679	3.7423	0.4479	0.7604	0.9265
FedSTDN($\varphi = 1$)	0.0701	0.3409	0.1085	0.1786	0.3301	0.2263	1.2211	2.2900	3.8406	0.4647	0.7397	0.9470
FedSTDN($\varphi = 10$)	0.0693	0.3398	0.1064	0.1718	0.3298	0.2246	1.1893	2.2228	3.8559	0.4620	0.7341	0.9413
FedSTDN($\varphi = 100$)	0.0673	0.3286	0.0789	0.1710	0.3273	0.1831	1.1004	2.1048	3.7306	0.4473	0.7183	0.9245

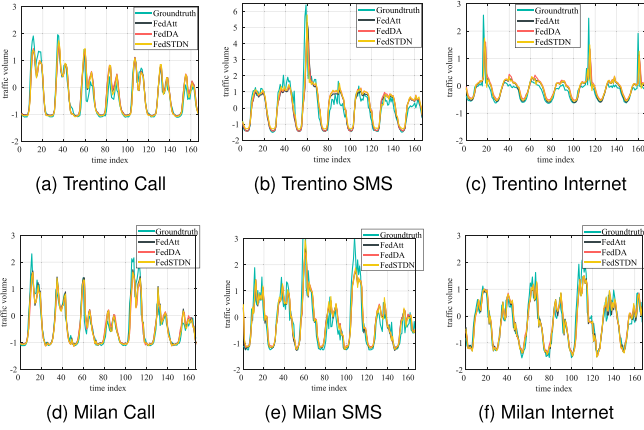


Fig. 6. Comparison of real values with predictions from different methods on two datasets.

neuron activation, and the prediction performance is improved by replacing the weight parameter with a univariate function parametrized as a spline function.

- 4) Compared with the proposed variant FedMLP, FedSTDN adopts a KAN layer instead of a fully connected layer. The fully connected layer utilizes MLP to learn the obtained features, but MLP has a fixed activation function for each neuron. The KAN layer has a learnable activation function for each weight. Such change makes KAN perform better than MLP.

To further evaluate the predictive ability of different methods, Fig. 6 shows the comparison results between the predicted values and the real values for different methods. Specifically, Fig. 6(a)–(c) present the prediction results on the Trentino dataset, while Fig. 6(d)–(f) present the prediction results on the Milan dataset. It can be seen from the figures that FedSTDN achieves more accurate prediction results across all three types of wireless traffic on both datasets.

D. Analysis of Communication Efficiency

In FL, communication resource is a crucial factor, and fewer communication rounds are often preferred. From [45], we have the total amount of bits that have to be uploaded and downloaded

by each client, expressed as

$$b^{\text{up/down}} \in O \left(N_{\text{iter}} \times f \times |\mathcal{W}| \times \left(H(\Delta \mathcal{W}^{\text{up/down}}) + \eta \right) \right), \quad (28)$$

where N_{iter} denotes the total number of training iterations performed by every client, f is the communication frequency, $|\mathcal{W}|$ means the size of the uploaded/downloaded model, $H(\Delta \mathcal{W}^{\text{up/down}})$ is the entropy of the weight updates exchanged during upload and download, respectively, and η is the inefficiency of the encoding. We assume the size of model and number of training iterations to be fixed, this leaves us with three options to reduce communication: 1) Reduce the communication frequency f , namely, the number of communication rounds between the client and the cloud server. 2) Reduce the entropy of the weight updates $H(\Delta \mathcal{W}^{\text{up/down}})$ via lossy compression schemes. 3) Reduce η by using more efficient encodings to communicate the weight updates.

Since the lossy compression and other effective encoding schemes are not considered, this work primarily focuses on reducing the communication overhead by minimizing the number of communication rounds f . In this subsection, we study the relationship between prediction accuracy and communication rounds for different methods, using training loss to represent prediction accuracy. The results are summarized in Fig. 7. From the figure, it can be seen that FedSTDN achieves lower loss on both datasets. More importantly, FedSTDN requires fewer communication rounds to achieve a certain prediction accuracy. Taking the Milan dataset as an example, after 30 communication rounds, the training losses of FedSTDN for Call, SMS, and Internet traffic are 0.1217, 0.2081, and 0.2204, respectively. In contrast, for FedAvg, FedAtt, FedDA, and FedMLP, the losses for Call traffic are 0.1966, 0.1963, 0.1666, and 0.1342, respectively; for SMS traffic, the losses are 0.2654, 0.2649, 0.2353, and 0.2199, respectively; and for Internet traffic, the losses are 0.2951, 0.2946, 0.2650, and 0.2334, respectively. Therefore, our proposed method is communication-efficient, which helps in performing FL tasks at the edge.

E. Analysis of Computational Complexity

In this subsection, according to [46], we analyze the computational complexity of the local models of FedDA, FedMLP, and FedSTDN, respectively. Firstly, we define the batch size as B ,

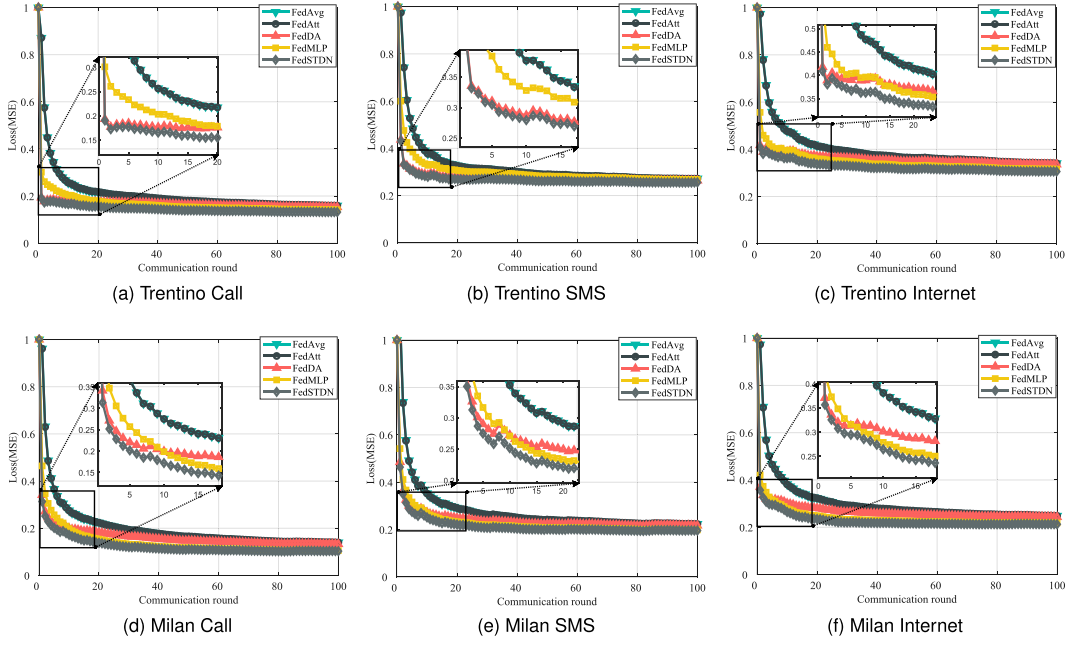


Fig. 7. The relationship between prediction performance and communication round.

sequence length as L , input dimension as d_{in} , output dimension as d_{out} , hidden layers dimension as d_h , convolution kernel size as k_{conv} . For KAN, we define the grid size and the piecewise polynomial order of the splines as G and v , respectively. The number of layers of MLP and KAN are defined as L_{MLP} and L_{KAN} , respectively.

For FedDA, its local model mainly consists of LSTM and MLP. The time complexity of the LSTM layer is given by $O(BL(d_{in} + d_h)d_h)$ and the time complexity of the MLP layer is expressed as $O(Bd_h^2L_{MLP})$. Therefore, the overall time complexity of the FedDA is

$$O(BL(d_{in} + d_h)d_h + Bd_h^2L_{MLP}). \quad (29)$$

For the space complexity, LSTM and MLP are given by $O(Bd_h + 4d_h(d_{in} + d_h))$ and $O(Bd_h^2L_{MLP})$, respectively. Thus, the space complexity of FedDA is

$$O(Bd_h + 4d_h(d_{in} + d_h) + Bd_h^2L_{MLP}). \quad (30)$$

For FedMLP, its local model consists of LSTM, 1D CNN and MLP. The time complexity of the 1D CNN layer is $O(BLd_{in}^2k_{conv})$. Thus, the time complexity of FedMLP is

$$O(BL(d_{in}^2k_{conv} + (d_{in} + d_h)d_h) + Bd_h^2L_{MLP}). \quad (31)$$

For the space complexity, 1D CNN is given by $O(Bd_{in}(2L - k_{conv} + 1) + k_{conv}d_{in}^2 + d_{in})$. Thus, the space complexity of FedMLP is

$$O(Bd_{in}(2L - k_{conv} + 1) + d_{in}(k_{conv}d_{in} + 1) + 4d_h(d_{in} + d_h) + Bd_h^2L_{MLP}). \quad (32)$$

FedSTDN has local model composed of LSTM, 1D CNN and KAN. The time complexity of the KAN layer is $O(aBd_h^2L_{KAN}G)$, where a denotes the number of computations required for each activation function. Therefore, the time

complexity of the FedSTDN is

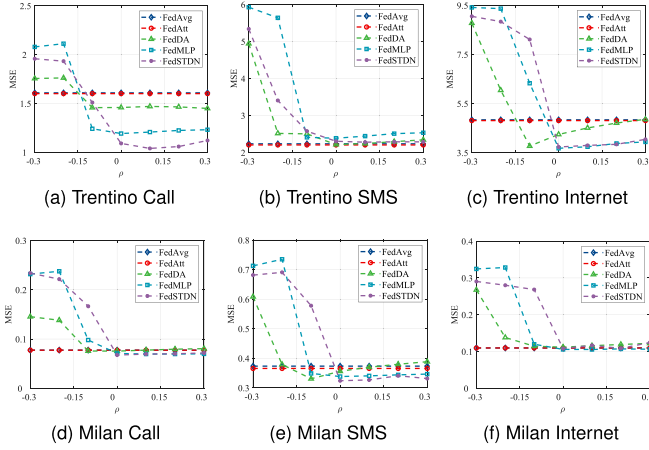
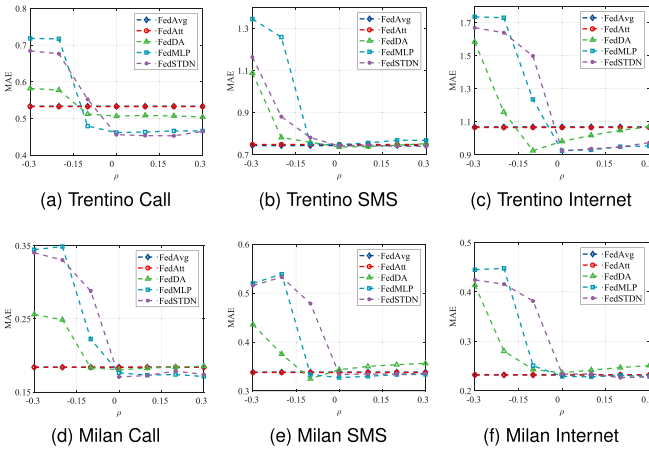
$$O(BL(d_{in}^2k_{conv} + (d_{in} + d_h)d_h) + aBd_h^2L_{KAN}G). \quad (33)$$

For space complexity, KAN is given by $O(Bd_h^2L_{KAN}(G + v))$. Thus, the space complexity of FedSTDN is

$$O\left(Bd_{in}(2L - k_{conv} + 2) + d_{in}(k_{conv}d_{in} + 1) + 4d_h(d_{in} + d_h) + Bd_h^2L_{KAN}(G + v)\right). \quad (34)$$

Through above analysis, it can be concluded that due to the introduction of the 1D CNN layer, the complexity of FedMLP and FedSTDN is higher than that of FedDA. FedSTDN replaces the MLP layer with the KAN layer, and the complexity of the KAN layer is related to the grid size G and the piecewise polynomial order of the splines v , which are generally greater than 1. However, FedDA and FedMLP typically require multiple MLP layers to improve prediction performance, while FedSTDN requires fewer KAN layers.

Due to the introduction of 1D CNN and KAN, the proposed FedSTDN may exhibit higher complexity compared to other methods, although it outperforms them in terms of both accuracy and communication efficiency. The increased complexity is a trade-off for achieving superior results, especially in tasks where high accuracy and fast convergence are critical. In the future, we will consider methods such as simplifying the KAN structure, adopting a hierarchical training strategy, and implementing adaptive grid strategies to reduce model complexity. Additionally, the trade-off between model complexity and prediction performance will also be an important aspect for us to analyze.

Fig. 8. The relationship between MSE and ρ on two datasets.Fig. 9. The relationship between MAE and ρ on two datasets.

F. The Effect of ρ on Prediction Performance

Since ρ is a task-dependent regularizer and it significantly impacts prediction performance, we present the prediction performance of different methods along with ρ and summarize the results in Figs. 8 and 9, which correspond to the MSE and MAE of different types of wireless traffic on the Milan and Trentino dataset, respectively. In both figures, ρ ranges from -0.3 to 0.3 with a step size of 0.1 . From Figs. 8 and 9, we can see that the MSE and MAE results for FedAvg and FedAtt remain constant for different types of wireless traffic, as they are not affected by ρ . For FedDA, FedMLP, and FedSTDN, as ρ increases, the MSE and MAE values first rapidly decrease and then slowly increase. This is because when ρ is too small, the model is too complex and prone to memorizing noise in the training data, resulting in poor performance. As ρ gradually increases, the constraints on the model increase, making the training error and test error more consistent. However, when ρ is too large, the model becomes overly constrained and cannot fully learn the patterns and features in the data, leading to suboptimal performance. Despite the significant influence of ρ

on prediction performance, FedSTDN can achieve lower MSE and MAE values compared to other baseline methods.

G. The Effect of R on Prediction Performance

The number of regions R also affects the final prediction performance. Setting R too small results in fewer traffic patterns, which cannot fully capture the spatial-temporal dependencies of the traffic, leading to poor performance. Conversely, setting R too large can cause the model to overfit, also resulting in suboptimal performance. Therefore, in this subsection, we study the relationship between the number of regions and the prediction performance of FedSTDN, and demonstrate the obtained MSE and MAE results in Fig. 10. Specifically, we consider three cases: $R = 1$, $R = 16$, and $R = 32$. It is worth noting that $R = 1$ means no clustering strategy is adopted in the prediction. Additionally, to make the comparison clearer, the MSE and MAE results for $R = 16$ and $R = 32$ are normalized based on the results for $R = 1$. It can be observed from the figure that the choice of R has different impacts on the prediction performance of FedSTDN. In most cases, introducing the clustering strategy indeed leads to lower prediction errors. Specifically, for the Milan dataset, FedSTDN achieves significant performance improvements in Call, SMS, and Internet traffic when the number is 16 or 32, with the normalized MSE improving by up to 11.32% and the normalized MAE by up to 10.80% for Internet traffic. For the Trentino dataset, introducing the clustering strategy also results in better prediction performance compared to not introducing it, especially for Internet traffic, where the normalized MSE is improved by up to 4.54% and the normalized MAE by up to 3.14%.

H. The Effect of KAN Parameters on Prediction Performance

Since this work adopts the KAN layer instead of the traditional fully connected layer to achieve better prediction performance, we also study the important parameters in the KAN layer. We select the grid size G and the order of the piecewise polynomial v as the subjects of our study.

The grid size G in KAN is primarily used to represent the upper bound of the error in approximation theory [38].

Lemma 1 (Kolmogorov-Arnold theorem (KAT)): Let $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Suppose that a function $f(\mathbf{x})$ admits a representation

$$f = (\Phi_{L_d-1} \circ \Phi_{L_d-2} \circ \dots \circ \Phi_1 \circ \Phi_0) \mathbf{x}, \quad (35)$$

where L_d denotes the depth of the KAN, $\circ$ represents the composite function operation. Each one of the $\Phi_{l,i,j}$ is $(v+1)$ -times continuously differentiable. Then there exists a constant C depending on $f(\mathbf{x})$ and its representation, such that we have the following approximation bound in terms of the grid size G : there exists v -th order B-spline functions $\Phi_{l,i,j}$ such that for any $0 \leq m \leq v$, we have the bound

$$\|f - (\Phi_{L_d-1}^G \circ \dots \circ \Phi_1^G \circ \Phi_0^G) \mathbf{x}\|_{C_m} \leq CG^{-v-1+m}. \quad (36)$$

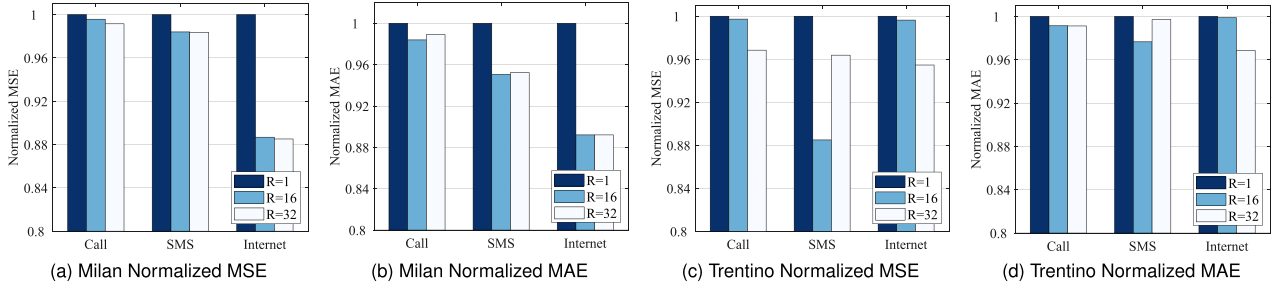


Fig. 10. The relationship between prediction performance and the number of regions.

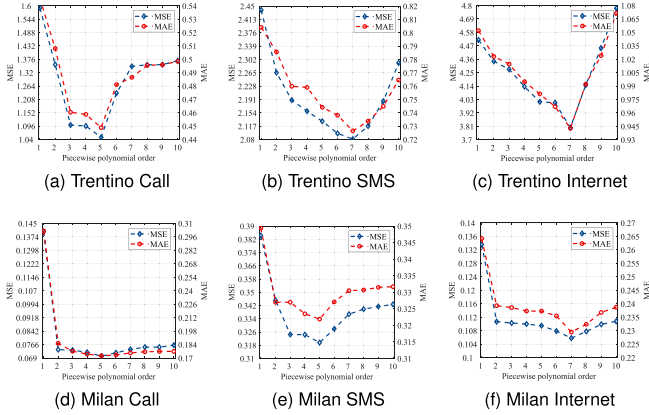


Fig. 11. The relationship between prediction performance and grid size.

Here we adopt the notation of C^m -norm measuring the magnitude of derivatives up to order m :

$$\|g\|_{C^m} = \max_{| \zeta | \leq m} \sup_{x \in [0,1]^n} |D^\zeta g(x)|. \quad (37)$$

The bound (36) indicates that as G increases, the approximation error decreases, thereby improving the model's accuracy. However, the number of parameters of KAN can be expressed as $O(d_h^2 L_{KAN}(G + v))$. We can see that when G is too large, the number of parameters in the model becomes excessive, which may affect the model's generalization ability and lead to performance degradation. Therefore, for different datasets, the optimal value of G varies to achieve outstanding performance. Accordingly, we study the impact of different G values on prediction performance, as shown in Fig. 11. As shown in the figure, the trends of MSE and MAE for Call, SMS, and Internet traffic with respect to grid size are basically consistent across different datasets, generally showing a trend of first decreasing and then slowly increasing. Additionally, for the Milan dataset, when the grid size is 5, the prediction performance for Call and SMS traffic reaches the optimal values, with MSE of 0.0701 and 0.3196, and MAE of 0.1724 and 0.3219, respectively. When the grid size is 7, the prediction performance for Internet traffic is optimal, with MSE and MAE of 0.1058 and 0.2295, respectively. For the Trentino dataset, when the grid size is 5, the prediction performance for Call traffic is optimal, with MSE and MAE of 1.0494 and 0.4489, respectively. When the grid size is 7, SMS and Internet traffic get well performance, with MSE of 2.0813 and 3.7999, and MAE of 0.7265 and 0.9428, respectively.

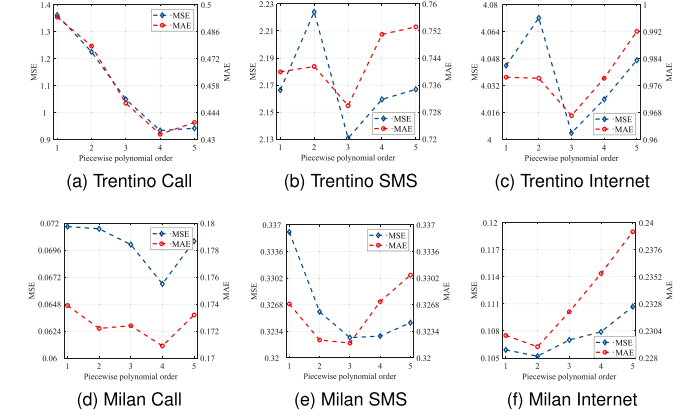


Fig. 12. The relationship between prediction performance and piecewise polynomial order.

The piecewise polynomial order v refers to the degree of the piecewise polynomial in approximation theory. It defines the maximum degree of the polynomial for each segment in the piecewise polynomial function, primarily affecting the smoothness of the B-spline function. From (36), it can be seen that as v increases, the B-spline function becomes smoother, which can represent finer features and further improve prediction accuracy. However, when v is too large, it may lead to an overly complex model, thereby increasing the risk of overfitting. Therefore, v needs to be chosen based on the specific task and the characteristics of the dataset to achieve the optimal performance. Accordingly, we study the impact of different v values on prediction performance, as shown in Fig. 12. As shown in the figure, the trends of MSE and MAE for different types of data with respect to v are generally consistent, showing a trend of first decreasing and then increasing. For the Milan dataset, the optimal v values for Call, SMS, and Internet traffic are 4, 3, and 2, respectively. The corresponding MSE values at these optimal v values are 0.0666, 0.3226, and 0.1052, and the corresponding MAE values are 0.1709, 0.3219, and 0.229, respectively. For the Trentino dataset, the optimal v values for Call, SMS, and Internet traffic are 4, 3, and 3, respectively. The corresponding MSE values at these optimal v values are 0.9336, 2.1307, and 4.0039, and the corresponding MAE values are 0.4328, 0.7299, and 0.9671, respectively.

VI. CONCLUSION

In this paper, we investigated the WTP problem under the FL strategy and proposed a novel framework called FedSTDN. To address the issues of complex spatial-temporal characteristics caused by user mobility and the data heterogeneity from various BSs, we first adopted a data augmentation-based iterative clustering strategy. In this strategy, each client user shares a small augmented dataset with the cloud server, which then trains a quasi-global model and distributes it to the clients. The cloud also groups the cells covered by different BSs into different regions based on the geographic location information and augmented data. Secondly, to improve the prediction performance of the local model, we proposed a prediction model based on 1D CNN and LSTM to capture the short- and long-term dependencies of the traffic. Thirdly, to further enhance the prediction performance, we adopted the novel KAN layer to replace the traditional MLP layer. Furthermore, we used a dual attention-based model aggregation method to aggregate model parameters by focusing on the contributions of the cell-level models, the region-level models, and the quasi-global model. Finally, we validated the effectiveness and efficiency of FedSTDN on two real-world datasets. Compared to the well-performing baseline, the proposed FedSTDN achieves up to 32.83% and 24.30% improvements in Mean Square Error (MSE) and Mean Absolute Error (MAE) on the Milan dataset, respectively. For the Trentino dataset, FedSTDN achieves up to 17.25% and 5.86% improvements in MSE and MAE, respectively.

REFERENCES

- [1] S. P. Sone, J. J. Lehtomäki, and Z. Khan, "Wireless traffic usage forecasting using real enterprise network data: Analysis and methods," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 777–797, 2020.
- [2] Y. Xu, F. Yin, W. Xu, J. Lin, and S. Cui, "Wireless traffic prediction with scalable Gaussian process: Framework, algorithms, and verification," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1291–1306, Jun. 2019.
- [3] Y. Fu, S. Wang, C.-X. Wang, X. Hong, and S. McLaughlin, "Artificial intelligence to manage network traffic of 5 G wireless networks," *IEEE Netw.*, vol. 32, no. 6, pp. 58–64, Nov./Dec. 2018.
- [4] M. Li, Y. Wang, Z. Wang, and H. Zheng, "A deep learning method based on an attention mechanism for wireless network traffic prediction," *Ad Hoc Netw.*, vol. 107, 2020, Art. no. 102258.
- [5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, PMLR, 2017, pp. 1273–1282.
- [6] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
- [7] M. B. Mollah, H. Wang, M. A. Karim, and H. Fang, "mmWave enabled connected autonomous vehicles: A use case with V2V cooperative perception," *IEEE Netw.*, vol. 38, no. 6, pp. 485–492, Nov. 2024.
- [8] Y. Fu, Z. Li, S. Liu, C. Li, F. R. Yu, and N. Cheng, "An incentive mechanism for long-term federated learning in autonomous driving," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 15642–15655, May 2024.
- [9] M. B. Mollah, M. A. K. Azad, and Y. Zhang, "Secure targeted message dissemination in IoT using blockchain enabled edge computing," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5389–5400, Aug. 2024.
- [10] L. Zhang, C. Zhang, and B. Shihada, "Efficient wireless traffic prediction at the edge: A federated meta-learning approach," *IEEE Commun. Lett.*, vol. 26, no. 7, pp. 1573–1577, Jul. 2022.
- [11] M. Kumari, Z. Ulmas, R. Suseendra, N. Ramesh, J. Venkata, and Y. A. B. Ebiary, "Utilizing federated learning for enhanced real-time traffic prediction in smart urban environments," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 2, pp. 639–651, 2024.
- [12] M. Xia, D. Jin, and J. Chen, "Short-term traffic flow prediction based on graph convolutional networks and federated learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 1191–1203, Jan. 2023.
- [13] X. Yuan et al., "FedSTN: Graph representation driven federated learning for edge computing enabled urban traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 8, pp. 8738–8748, Aug. 2023.
- [14] C. Zhang, S. Dang, B. Shihada, and M.-S. Alouini, "Dual attention-based federated learning for wireless traffic prediction," in *Proc. IEEE INFOCOM 2021 IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [15] C. Qiu, Y. Zhang, Z. Feng, P. Zhang, and S. Cui, "Spatio-temporal wireless traffic prediction with recurrent neural network," *IEEE Wireless Commun. Lett.*, vol. 7, no. 4, pp. 554–557, Aug. 2018.
- [16] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, "Citywide cellular traffic prediction based on densely connected convolutional neural networks," *IEEE Commun. Lett.*, vol. 22, no. 8, pp. 1656–1659, Aug. 2018.
- [17] Y. Zhu, L. Feng, F. Zhou, and W. Li, "An adaptive ensemble learning paradigm with spatial-temporal feature extraction for wireless traffic prediction," *IEEE Trans. Netw. Service Manag.*, early access, Dec. 23, 2024, doi: 10.1109/TNSM.2024.3522115.
- [18] Z. Wang and V. W. S. Wong, "Bayesian meta-learning for adaptive traffic prediction in wireless networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 6620–6633, Jun. 2024.
- [19] W. Shen, H. Zhang, S. Guo, and C. Zhang, "Time-wise attention aided convolutional neural network for data-driven cellular traffic prediction," *IEEE Wireless Commun. Lett.*, vol. 10, no. 8, pp. 1747–1751, Aug. 2021.
- [20] H. Zheng, F. Lin, X. Feng, and Y. Chen, "A hybrid deep learning model with attention-based conv-LSTM networks for short-term traffic flow prediction," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 11, pp. 6910–6920, Nov. 2021.
- [21] A. Mehrabian, S. Bahrami, and V. W. Wong, "A dynamic Bernstein graph recurrent network for wireless cellular traffic prediction," in *Proc. ICC 2023 IEEE Int. Conf. Commun.*, 2023, pp. 3842–3847.
- [22] Z. Li, Y. Fu, P. Zhao, and C. Li, "A dynamic spatiotemporal prediction method for urban network traffic," in *Proc. IEEE 96th Veh. Technol. Conf.*, 2022, pp. 1–5.
- [23] L. Yu et al., "STEP: A spatio-temporal fine-granular user traffic prediction system for cellular networks," *IEEE Trans. Mobile Comput.*, vol. 20, no. 12, pp. 3453–3466, Dec. 2021.
- [24] F. Sun et al., "Mobile data traffic prediction by exploiting time-evolving user mobility patterns," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4456–4470, Dec. 2022.
- [25] P. Li, C. Guo, Y. Xing, Y. Shi, L. Feng, and F. Zhou, "Distributed core network traffic prediction architecture based on vertical federated learning," in *Proc. Int. Conf. Comput. Eng. Netw.*, Springer, 2023, pp. 230–237.
- [26] X. Shen, J. Chen, S. Zhu, and R. Yan, "A decentralized federated learning-based spatial-temporal model for freight traffic speed forecasting," *Expert Syst. With Appl.*, vol. 238, 2024, Art. no. 122302.
- [27] T. Qi, L. Chen, G. Li, Y. Li, and C. Wang, "FedAGCN: A traffic flow prediction framework based on federated learning and asynchronous graph convolutional network," *Appl. Soft Comput.*, vol. 138, 2023, Art. no. 110175.
- [28] L. Liu et al., "Multilevel federated learning-based intelligent traffic flow forecasting for transportation network management," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1446–1458, Jun. 2023.
- [29] J. Du et al., "MADDPG-based joint service placement and task offloading in MEC empowered airground integrated networks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 10600–10615, Mar. 2024.
- [30] J. Du et al., "Resource pricing and allocation in MEC enabled blockchain systems: An A3C deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 33–44, Jan./Feb. 2022.
- [31] F. Solat, T. Y. Kim, and J. Lee, "A novel group management scheme of clustered federated learning for mobile traffic prediction in mobile edge computing systems," *J. Commun. Netw.*, vol. 25, no. 4, pp. 480–490, Aug. 2023.
- [32] D. Kim, S. Shin, J. Jeong, and J. Lee, "Joint edge server selection and data set management for federated-learning-enabled mobile traffic prediction," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 4971–4986, Feb. 2024.
- [33] H. P. Phyu, R. Stanica, and D. Naboulsi, "Multi-slice privacy-aware traffic forecasting at ran level: A scalable federated-learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 5038–5052, Dec. 2023.
- [34] C.-H. Lee, C.-R. Lin, and M.-S. Chen, "Sliding-window filtering: An efficient algorithm for incremental mining," in *Proc. 10th Int. Conf. Inf. Knowl. Manage.*, 2001, pp. 263–270.
- [35] F. Xu, Y. Li, H. Wang, P. Zhang, and D. Jin, "Understanding mobile traffic patterns of large scale cellular towers in urban environment," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 1147–1161, Apr. 2017.

- [36] H. Zhu, J. Xu, S. Liu, and Y. Jin, "Federated learning on non-IID data: A survey," *Neurocomputing*, vol. 465, pp. 371–390, 2021.
- [37] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [38] Z. Liu et al., "KAN: Kolmogorov-Arnold networks," 2024, *arXiv:2404.19756*.
- [39] A. N. Kolmogorov, *On the Representation of Continuous Functions of Several Variables by Superpositions of Continuous Functions of a Smaller Number of Variables*. Providence, RI, USA: American Mathematical Society, 1961.
- [40] S. Kaspour and A. Yassine, "Federated non-intrusive load monitoring for smart homes utilizing attention-based aggregation," in *Proc. 2023 IEEE Int. Conf. Ind. 4.0, Artif. Intell., Commun. Technol.*, 2023, pp. 208–213.
- [41] Z. Shan, "Federated client selection and attention-based aggregation algorithm on non-IID data," in *Proc. 2024 3rd Int. Conf. Cloud Comput., Big Data Appl. Soft. Eng.*, 2024, pp. 111–114.
- [42] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," *Sci. Data*, vol. 2, no. 1, pp. 1–15, 2015.
- [43] S. Ji, S. Pan, G. Long, X. Li, J. Jiang, and Z. Huang, "Learning private neural language modeling with attentive aggregation," in *Proc. 2019 Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.
- [44] J. Gan, W. Wang, and K. Lu, "A new perspective: Recognizing online handwritten Chinese characters via 1-dimensional CNN," *Inf. Sci.*, vol. 478, pp. 375–390, 2019.
- [45] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-iid data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
- [46] P. Freire, S. Srivallapanondh, A. Napoli, J. E. Prilepsky, and S. K. Turitsyn, "Computational complexity evaluation of neural network applications in signal processing," 2024, *arXiv:2206.12191*.



Zhenyu Li (Graduate Student Member, IEEE) received the BEng degree in college of information and control engineering from the Xi'an University of Architecture and Technology, XAUAT, China, in 2022. He is currently working toward the PhD degree with the School of Telecommunication Engineering of Xidian University. His currently research interests include wireless traffic prediction, performance analysis, and resource allocation for ISCC in IoV.



Yuchuan Fu (Member, IEEE) received the PhD degree from the School of Telecommunications Engineering, Xidian University, Xi'an, China, in 2020. She is currently an associate professor with the State Key Lab of ISN, School of Telecommunication Engineering, Xidian University. From 2018 to 2019, she was a joint PhD student with Carleton University, Ottawa, ON, Canada. Her current research interests include algorithm design in vehicular networks and autonomous driving.



Mengqiu Tian (Student Member, IEEE) received the BE degree and the MS degree from the Department of Information Science and Technology, Nanjing Forestry University, Nanjing, China, in 2018 and 2021, respectively. She is currently working toward the PhD degree in information and communication engineering with Xidian University, China. Her current research interests include B5G/6 G networks, vehicular networks, and on-demand resource scheduling.



ad hoc networks, and wireless sensor networks.

Changle Li (Senior Member, IEEE) received the PhD degree in communication and information system from Xidian University, China, in 2005. He conducted his postdoctoral research in Canada and the National Institute of information and Communications Technology, Japan, respectively. He had been a visiting scholar with the University of Technology Sydney and he is currently a professor with the State Key Laboratory of Integrated Services Networks, Xidian University. His research interests include intelligent transportation systems, vehicular networks, mobile



F. Richard Yu (Fellow, IEEE) received the PhD degree in electrical engineering from the University of British Columbia in 2003. From 2002 to 2006, he was with Ericsson in Lund, Sweden and a start-up in California, USA. He joined Carleton University in 2007, where he is currently a professor. His research interests include connected/autonomous vehicles, security, artificial intelligence, blockchain, and wireless cyber-physical systems. He was with the editorial boards of several journals, including co-editor-in-chief for Ad Hoc & Sensor Wireless Networks, Lead Series editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, and IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING. He was with the Technical Program Committee co-chair of numerous conferences. He was the recipient of the IEEE TCGCC Best Journal Paper Award in 2019, Distinguished Service Awards in 2019 and 2016, Outstanding Leadership Award in 2013, Carleton Research Achievement Awards in 2012 and 2021, Ontario Early Researcher Award (formerly Premiers Research Excellence Award) in 2011, Excellent Contribution Award at IEEE/IFIP TrustCom'10, Leadership Opportunity Fund Award from Canada Foundation of Innovation in 2009 and Best Paper Awards at IEEE ICNC'18, VTC'17 Spring, ICC'14, Globecom'12, IEEE/IFIP TrustCom'09 and Int'l Conference on Networking 05. He has been named in the Clarivate Analytics list of "Highly Cited Researchers" in 2019 and 2020. He is an IEEE distinguished lecturer of both Vehicular Technology Society (VTS) and Comm. Society. He is an elected member of the Board of Governors of the IEEE VTS and editor-in-chief for IEEE VTS Mobile World newsletter. Dr. Yu is a registered professional engineer in the province of Ontario, Canada, and a fellow of IET, and Engineering Institute of Canada.



Nan Cheng (Senior Member, IEEE) received the PhD degree from the Department of Electrical and Computer Engineering, University of Waterloo in 2016, and the BE degree and the MS degree from the Department of Electronics and Information Engineering, Tongji University, Shanghai, China, in 2009 and 2012, respectively. He was a postdoctoral fellow with the Department of Electrical and Computer Engineering, University of Toronto, from 2017 to 2018. He is currently a professor with State Key Lab of ISN, and with the School of Telecommunications Engineering, Xidian University, Shaanxi, China. His current research interests include space-air-ground integrated system, Big Data in vehicular networks, self driving system, performance analysis, MAC, opportunistic communication, and application of AI for vehicular networks.